PortalOps

The nervous system for *Portaldot*.

Today: signed, retried, replayable events for any contract. Next: the layer that AI agents, games and security tools build on.

Portaldot Hacker House · Track: Modular Cloud & Middleware

WHERE PORTALDOT IS HEADING

The next apps on Portaldot will be *agents and games*.

AI agents

They act on-chain and react to what happens on-chain, all day, with no human watching.

Games

Every move, quest and reward is an event. Games live and die by how fast they can react.

Builders

They need to ship contracts fast, and know right away if something is going wrong.

All three need the same thing: to know, reliably and in real time, **what just happened on the chain**. That is the layer PortalOps provides.

THE GAP TODAY

But a contract's events are *unreadable* out of the box.

32 contracts

225 events

65 kinds

0 readable without an ABI

raw hex · reorganised blocks · servers that go down

Portaldot V3.0 testnet · blocks 91,528 to 145,105 · measured 2026-10-22 with PortalOps' public explorer (no ABIs registered, so every event is raw)

WHAT PORTALOPS IS TODAY

Watch. Decode. Deliver. *Recover.*



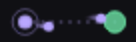
Watch

Follows the chain block by block and spots logs from the contracts you register.



Decode

Turns raw logs into named, typed events using the contract ABI. Stored exactly once.



Deliver

Posts each event to your endpoint as a signed webhook and records what came back.



Recover

Retries with backoff when you are down. Replays any event later, without touching the chain.

Dashboard · No-signup playground · REST API · TypeScript SDK · Claude (MCP)

PROOF IT WORKS

Not a mock-up. *Live on the Portaldot testnet.*

PORTALQUEST · DEPLOYED BY US

0x274F8AbeCAa7FE3772A2cB92F67F2ed4A64bc649

Block 145,098 · real events: **QuestCreated**, **QuestCompleted**, **RewardIssued**

PORTALPING · DEPLOYED BY US

0x6a8705e4621dd87a261b1c5f97b73871616537ec

Block 152,078 · live **Ping** events, captured and delivered on demand

captured → decoded → delivered ✓ signature verified → fails (500) → retried → replayed ✓

170+ tests, incl. real testnet logs

HMAC-signed

SSRF-safe

AES-256-GCM secrets

Reorg-aware, honest finality

AI AGENTS, ALREADY

An agent can *operate PortalOps* today.

Ten Claude (MCP) tools

- Ask in plain English: *"Watch my Quest contract and send its events to the test receiver."*
- The agent checks the project, registers the contract with the right ABI, creates the webhook and sends a signed test.
- Signing secrets are written to a local file. They never enter the chat.

Why this matters

- Setting up chain monitoring is a conversation, not a project.
- Agents get a safe, scoped way in: project-limited keys, read or write.
- It is the first step toward agents that **watch** the chain, not just write to it.

ON THE ROADMAP

THE FUTURE: AGENTS ON PORTALDOT

PortalOps becomes the *eyes of every agent* on the chain.

Events as triggers

On-chain events flow straight into an agent's loop as signed, decoded messages.

Finality-gated actions

An agent acts only on *finalized* data, so it never moves money on a reorganised block.

Crash recovery

If an agent goes down, replay catches it up exactly, with nothing missed or doubled.

Agent audit trail

Watch what each agent's wallet does on-chain, as readable events with full history.

`event on chain` → `PortalOps: decoded, finalized` → `agent decides` → `contract call` → `PortalOps confirms`

PART BUILT

THE FUTURE: BUILDING ON PORTALDOT, FAST

From idea to a *live game contract in minutes*.

`describe it` → `template` → `compile` → `deploy` → `watch` → `webhooks live`

Built today

- Game-shaped starter contracts: a quest board, a bounty board and a token, each with ABI and a ready receiver.
- Scripts that compile and deploy a contract, with gas and balance checks for Portaldot's weight-based costs, then the dashboard watches it once you paste the address and ABI.

Next

- One command, or one sentence to Claude, that deploys **and** starts watching.
- More templates for common game and agent patterns, with events already wired to webhooks.

ON THE ROADMAP

THE FUTURE: SECURITY THAT NEVER SLEEPS

Real-time *security watching* for every contract.

- unlimited approvals
- ownership transferred
- mint / burn spikes
- repeated failed calls
- admin functions called

How it works

- Alert rules run over the same live event stream, as soon as an event is captured.
- Reorg- and finality-aware, so alerts are not false alarms.
- Delivered to Slack, Discord, email, a webhook, or an agent that can react.

Built on what exists

- The event stream and decoding.
- Automations that already route events by rule.
- The delivery engine with signing, retries and replay.

WHY PORTALDOT SHOULD BACK THIS

Make Portaldot the chain where *agents and games ship fastest*.

What it does for the ecosystem

- Every developer and every agent gets observability from day one, instead of rebuilding it.
- Faster building and safer contracts means more apps, and apps that last.
- A visible reason to build here: events are readable, reliable and secure by default.

Built around how Portaldot really behaves

- pallet-revive EVM: account mapping handled and documented.
- Only ~256 recent blocks are served by number, so backfill uses log queries.
- Finality comes from the node's finalized head, never guessed.
- Weight-based gas shaped our deploy tooling.

ROADMAP

The core is live. *Support turns it into the platform.*

Now · live

- Watch, decode, deliver, recover
- Dashboard, playground, REST API
- Claude (MCP) setup
- Live on the testnet, open source

Next

- SDK on npm
- Security alert rules
- Agent triggers and audit trail
- One-step deploy and watch

Later

- Native Rust / ink! adapter
- Hosted multi-tenant service
- More Portaldot networks and chains
- Chat, email and queue targets

PortalOps

Watch any contract. *Never miss an event.*

LIVE APP	portalops.onrender.com
TRY IT, NO SIGNUP	portalops.onrender.com/playground
CODE & DOCS	github.com/nofazechris/portalops

PortalQuest 0x274F8Abe…4bc649 · PortalPing 0x6a8705e4…6537ec